Gmail SHREYAS S BAGI <shreyassbagi@gmail.com>

LEARNING BASIC : PERFORMANCE WORK MODELLING WORK

SHREYAS S BAGI <shreyassbagi@gmail.com> 8 June 2026 at 07:47
To: ssbagi@ibm.com

Hi,

AI is helpful only for beginners <4 year experience of work ---> Like me working Hardware Semiconductor Domain.

Only the people who have written code, understood hardware or debugged or have done more mistakes and corrected then only using AI helps to do the work.

I am at this stage learn, work, make mistakes, correct them and Seniors provide feedback for correction for me and stuff.

Based on working or reading on the same things this method like everyday or every time we know where are the issues at least which hardware unit(block unit) to modify or improve the things.

So below the RTL DCache and SystemC DCacche mimic the behaviour exactly so comparison of hits and misses gives a proper Validation or Confidence that it is working as per Specification and stuff.

The Testcases are already defined and same we can just reuse for Correctness measurements.

It's just a perspective gives for improvements just like showed in GEM5 simulators also. A short root cause mechanism. Not every time a big change can happen in each and every model or hardware block all the time.

GitHub Link :
This is same like the STA report : number of clocks cells count on each path I made during internship. So every time we have fixes or to reduce the distortion there was a pattern. So based on this as level 0 debug I made it because I was giving the fixes for few cell paths were common in the timing reports because I was knowing what I am doing and what has to be done and corrected. The main root parent path cells of corrected then many paths would be impacted also.

It was same thing repeated for few months so a pattern got formed.

Improving Modelling Quality, Validation, and Correlation

Going Stage by Stage : Proper order lets it take time for the completion **since it is at Iteration1 or Iteration2 stage only**.

1. Objective

Strengthen the modelling workflow by introducing **structured validation**, **architecture-intent-aware testing**, and **early DV–Model correlation**.
This improves correctness, reduces debug time, and increases confidence before PD.

2. Roles & Responsibilities

2.1 Senior / Lead Modelling Engineers

- Own **architecture model design**
- Define **modelling boundaries** (architectural vs microarchitectural)
- Specify **timing intent, arbitration, replacement rules**
- Provide **test categories, TG patterns, and validation strategy**
- Guide juniors on correctness and behaviour

2.2 Junior Modelling Engineers (That is me)

- Understand architecture
- Read and understand model code
- Validate behaviour
- Run traces and analyze mismatches
- Write helper functions, tests, checkers
- Add TG patterns
- Ensure correlation with RTL

Not expected: deciding architecture, modelling boundaries, timing intent, arbitration, replacement policies.

3. Why Structured Validation Is Needed

Current validation is functional but not systematic.
Adding structure improves:

- Behavioural correctness
- Debug clarity
- Architecture-intent coverage
- Repeatability
- Early mismatch detection

4. Proposed Validation Enhancements

4.1 Traffic Generator (TG) Scenarios

Use TGs to exercise:

- Load/store patterns
- Bank conflicts
- Arbitration
- Pipeline corner cases

4.2 Categorized Test Suites

- Basic functionality
- Corner cases
- Stress/random
- Performance loops

- Ordering/consistency tests

4.3 Architecture-Intent-Aware Checkers

Checkers that validate:

- Ordering rules
- Bank behaviour
- Latency expectations
- Protocol correctness

5. DV–Modelling Correlation (Early-Stage)

DV already runs **20M–40M trace workloads**.
Proposal: add a **lightweight correlation stage** in modelling.

What to compare

- Event counts
- Cycle counts
- Bank conflicts
- Pipeline stalls
- Throughput metrics

Acceptable correlation target

70–75% correlation is sufficient for early modelling stages.

This provides confidence before synthesis/PD and reduces late-stage surprises.

6. Prototype C++ Model for Early Validation

A small, abstract C++ model can be used to:

- Validate pipeline behaviour
- Count events
- Compare against DV
- Avoid huge VCD/log dumps
- Print only summary counters (heartbeat-style)

Useful at:

- Post-RTL freeze
- Pre-synthesis
- GLS
- Pre-PD signoff

7. Expected Outcomes

- Cleaner modelling
- Faster debug cycles

- Higher correctness confidence
- Early detection of mismatches
- Better alignment with RTL/DV
- Predictable performance modelling accuracy

8. Final Summary

This proposal introduces a **structured, scalable, and architecture-aligned validation flow** for modelling teams. It clarifies junior vs senior responsibilities, improves test quality, and adds early DV correlation to ensure correctness before PD.

They are matching : The number of hits, misses, hit and miss ratio. ----------> These are basic ones.

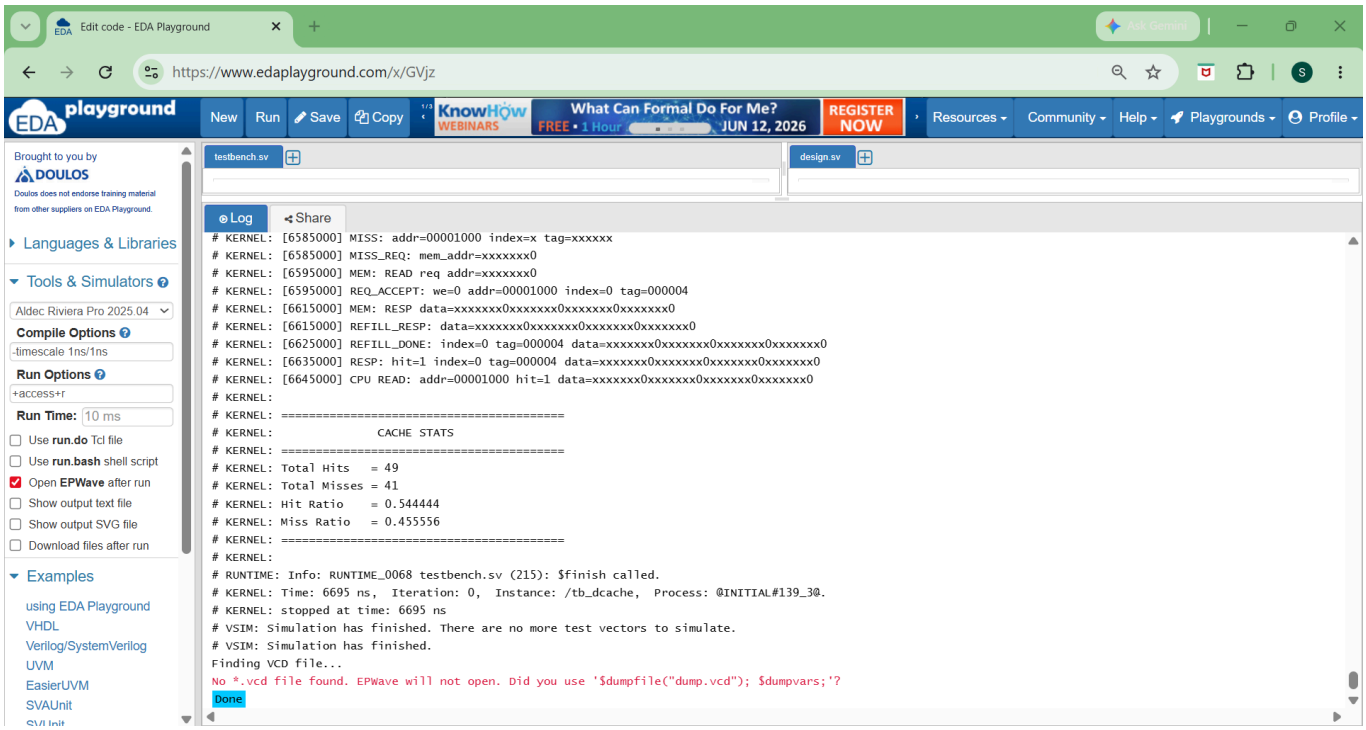
Now let us compare the Logs just for cross verifying things for a few basic Unit level Transactions : I mean like the Behaviour wise Standalone Test Cases and stuff.

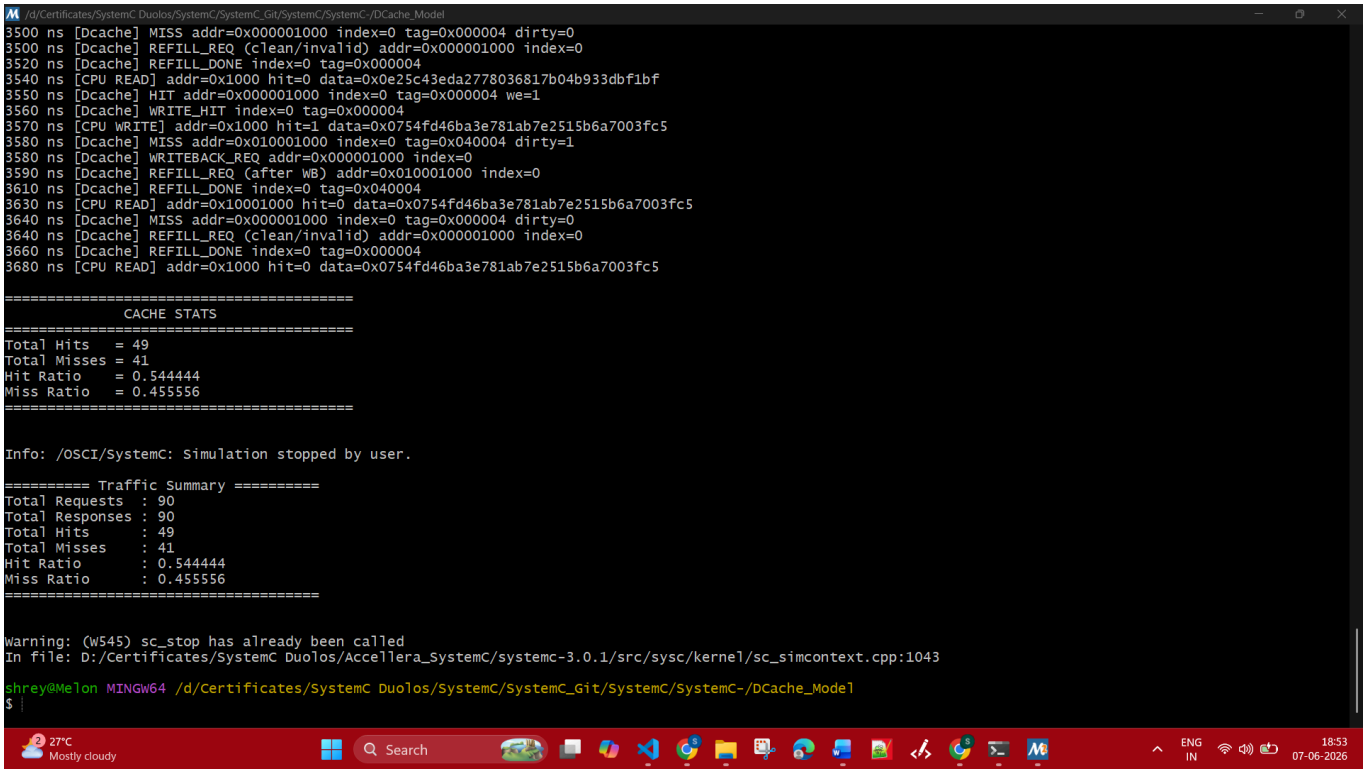
Based on reading and working this method like everyday or everytime we know where are the issues at least which to unit to modify or improve the things. It's just a perspective gives for improvements just like showed in GEM5 simulators also.

Why ?

- **To have basic behaviour is to do the same or not.** The Performance Modelling team comes up with the Models and the RTL and DV do the work. Just how much it is accuray done even **if we have done >75% meeting is also fine**.
- **Standalone behaviour is merely telling the unit wise they are working fine and stuff. Short Traces the behaviour wise they are correct.** (Kind of subunit or IPs)
- Now when we integrate into the whole CPU pipeline or Main block there might be numerous reasons not matching ----------> from pipe_viewer(vcd dump) we come to know. The DV stats do not match the Golden Model developed by the Performance Modelling team.
- Like how seniors teaching the debugging basics since we run more workloads or something. Basic one is stats comparison.
- Basic Few Unit level transactions between the SystemC/C++ to RTL/DV for a few of them. It gives us the confidence that it is working as per the specification and stuff.

The Stats from the DV side :





[Quoted text hidden]

7 attachments

- **directmap_cache_bkp.v**
  8K
- **DcacheSystemC.cpp**
  8K
- **SystemCMain.cpp**
  4K
- **Makefile**
  1K
- **DcacheSystemC.h**
  3K
- **TrafficGenerator.cpp**
  11K
- **TrafficGenerator.h**
  3K